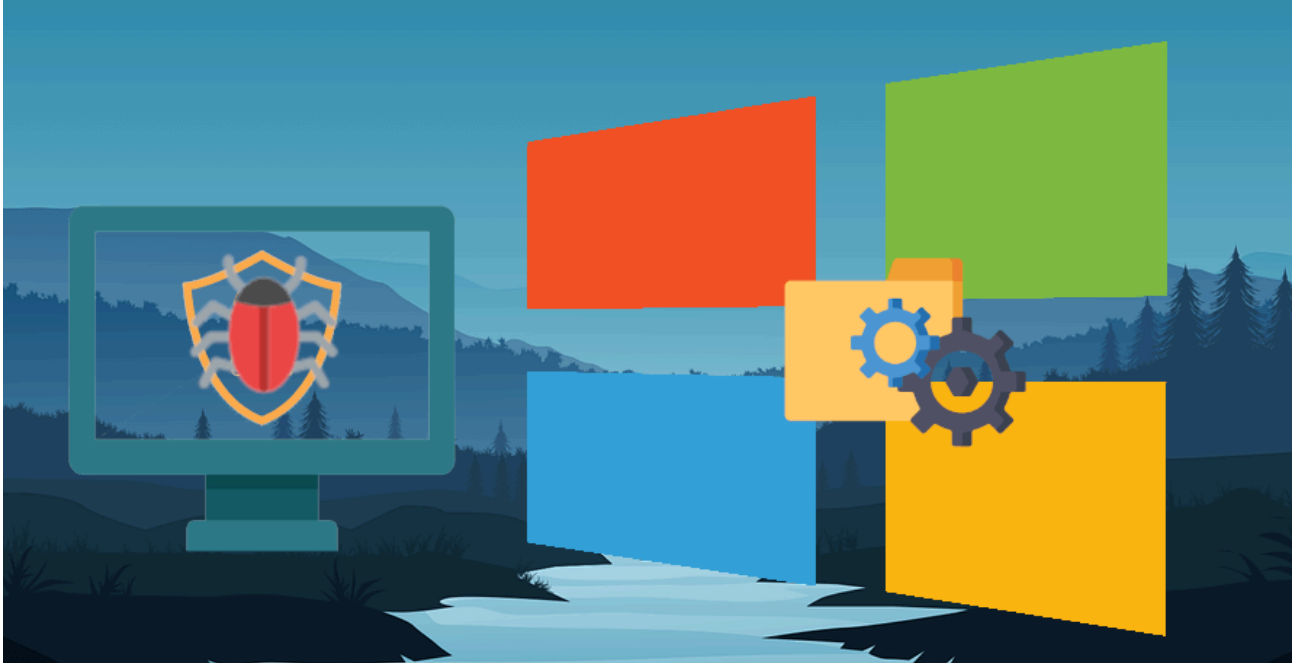


Unquoted Service Path Privilege Escalation – Michael Waterman

 michaelwaterman.nl/2023/02/06/unquoted-service-path-privilege-escalation

Michael Waterman

February 6, 2023



Privilege escalation is a common attack vector used by attackers to gain higher levels of access to a computer system. One type of privilege escalation attack is the “*Unquoted Service Path*”, which is a vulnerability that exists in Windows operating systems. This type of attack can allow a threat actor to execute malicious code with elevated privileges, which can have serious consequences such as data theft or a complete system compromise. In this blog post, I’ll delve into the mechanics of Unquoted Service Path attacks and how they can be prevented. Understanding these types of attacks and how to defend against them is critical for protecting computer systems and sensitive data.

Understanding privilege escalation

Privilege escalation is a critical concept in computer security and refers to the act of a user, process or program gaining elevated access privileges to resources or sensitive data that are not normally available to them. This can happen either through exploitation of vulnerabilities in the system, configuration errors, or through exploiting weaknesses in the security policies and procedures. Understanding privilege escalation is important for both attackers and defenders, as attackers use it to gain unauthorized access to sensitive information, while defenders use it to identify and prevent these types of attacks. This article will provide an in-depth overview of privilege escalation regarding the use of a vulnerability known as an “*Unquoted Service Path*”, including its usage, how it works, and the measures that can be taken to prevent the misuse. By understanding privilege

escalation, organizations can better protect themselves against potential security threats and ensure the confidentiality and integrity of their information and systems.

Unquoted Service Paths

Unquoted Service Paths is a vulnerability that exists in the Windows operating system and refers to the practice of not enclosing the file path of a service executable in quotation marks. This can cause problems when the file path of the service executable contains spaces, as the operating system will look for files by traversing the file system before it looks in the specified path. This can lead to unintended execution of malicious files that are located in the current working directory and have the same name as the service executable.

For example, if a service has an unquoted path to "*C:\Program Files\Service\service.exe*", and a malicious file named "*service.exe*" is placed in the current working directory, the operating system, potentially, will execute the malicious file instead of the intended service executable. This can result in privilege escalation, as the malicious file will run with the same privileges as the service.

To prevent Unquoted Service Paths vulnerabilities, it is recommended that the file path of all service executables be enclosed in quotation marks. This will ensure that the operating system will always look for the service executable in the specified path, even if the path contains spaces. Additionally, organizations should conduct regular vulnerability scans and penetration tests to identify and remediate Unquoted Service Paths vulnerabilities.

It's also important to note that Unquoted Service Paths is just one type of privilege escalation vulnerability, and there are many other methods that attackers can use to gain elevated access privileges. However, by understanding and addressing Unquoted Service Paths vulnerabilities, organizations can take an important step towards improving their overall security posture and protecting themselves against potential security threats.

A practical example

In this section of the blog post, we will examine the impact of Unquoted Service Paths by providing a real-world example. We will use a custom Windows service that has already been built in C# to demonstrate how an Unquoted Service Path can lead to privilege escalation. By studying this example, you will gain a deeper understanding of how the vulnerability works and the steps you can take to prevent them in your own applications.

This example will focus on the consequences of having an Unquoted Service Path in a custom Windows service. I will show how an attacker can leverage this vulnerability to execute malicious code with elevated privileges. By the end of this example, you will have

a hands-on understanding of the risks posed by Unquoted Service Paths and how to guard against them in your own systems.

The lab setup

In this demonstration, we will use a single Windows 10 Enterprise machine that is fully updated at the time of writing. The custom service that we will examine is installed as part of a demo application in the following location: *"C:\Program Files\My Application Files\Service Files\unquotedService.exe"*.

The specific file that we will use during the demonstration is called *"ByYourCommand.exe"*. This file, when started, creates a named pipe that can be used to run any command as the system user. It's important to note that the original service runs as the local system account, which has elevated privileges on the system.

It's also important to note that the demo application and custom service have been created specifically for this demonstration and are not intended for use in a production environment. This lab setup provides a controlled environment for us to explore the impact of an unquoted service path and to demonstrate the steps that can be taken to prevent this type of vulnerability.

By using a fully updated Windows 10 Enterprise machine, a custom service that has been installed as part of a demo application, and a file named *"byYourCommand.exe"*, we can provide a realistic and easily reproducible example of an Unquoted Service Path vulnerability. This will allow you to gain a deeper understanding of this security risk and the measures you can take to protect against it in your own systems.

Identifying vulnerable services

One way to identify Unquoted Service Paths in a Windows system is to search the registry for paths that do not contain quotes around the "ImagePath" keys. This can be done using Windows Management Instrumentation (WMI) and a PowerShell script.

The following code uses WMI to retrieve information about all the services installed on a Windows machine, filters out services that are located in the *"C:\Windows"* directory and have quotes in their path, and then selects only the *"Name"* and *"Pathname"* properties of the remaining services:

```
Get-WMIObject win32_service | Where-Object {$_.pathname -notlike "*c:\windows\*" -and $_.pathname -notlike "\"*"} | select Name, pathname
```

PowerShell

This code works as follows:

1. `get-WMIobject win32_service` retrieves information about all the services installed on the Windows machine using WMI.
2. `Where-Object {$_.pathname -notlike "*c:\windows\*" -and $_.pathname -notlike "\""}` filters out services that are located in the “C:\Windows” directory and have quotes in their path. The `-notlike` operator is used to exclude services whose pathname does not match the specified pattern, have quotes in the pathname.
3. `select Name, pathname` selects only the “Name” and “Pathname” properties of the remaining services.

On the demo system this results in the detection of one vulnerable service.

By running this script, you can quickly identify services that have Unquoted Service Paths, which can be a security risk in your system. It’s important to note that this is just one way to identify Unquoted Service Paths, and other methods may exist.

Monitoring service behavior

To demonstrate the impact of Unquoted Service Paths, you can monitor how a service starts when its path in the registry does not contain quotes. One tool you can use to do this is [Process Monitor](#), a free tool from Microsoft Sysinternals.

Using Process Monitor, you can observe the file system and registry operations performed by your custom service and see all the directories and executables that it tries to find when starting up. This will show you the search path that the service uses, and can reveal potential security risks if any malicious executables are located in those directories.

To use Process Monitor, simply start the tool and then configure it to capture events from your custom service. You can do this by clicking the “*Filter*” button and selecting “*Process Name*” from the drop-down menu, then entering the name of the custom service. Then, start the service and observe the events that are captured by Process Monitor.

When taking this specific service into consideration, the filter criteria would contain the following:

Column	Relation	Value	Action
Process Name	Is	services.exe	Include
Result	Is	NAME NOT FOUND	Include

In the events list, you can see all the file system and registry operations performed by the service, including the path of the executable that the service is trying to start, as well as all the directories that it is searching along the way. To filter the unwanted noise, deselect the option to include registry information, this results in the following overview.

services.exe	880	CreateFile	C:\Program	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program.exe	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program Files\My	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program Files\My.exe	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program Files\My Application	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program Files\My Application.exe	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program Files\My Application Files\Service	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program Files\My Application Files\Service.exe	NAME NOT FOUND

When a service starts up and its registry path does not contain quotes, Windows uses a process known as path traversal to locate the executable that the service is supposed to run. This process can be seen in the events captured by Process Monitor.

During path traversal, Windows first checks the directory specified in the registry path for an executable with the same name as the service. If it doesn't find one, it adds ".exe" to the end of the directory name and checks again. If it still doesn't find an executable, Windows will repeat this process, adding ".exe" to the end of each directory name along the way, until it locates an executable or has checked all possible directories.

In this case, as seen in Process Monitor, Windows first checked the directory "C:\program". Since it didn't find an executable there, it added ".exe" to the end of the directory name and checked "c:\program.exe". It then moved on to check "C:\program files\my", followed by "C:\program files\my.exe". It repeats this behavior until it finds a match. As can be seen in the example picture "C:\Program Files\My Application Files\Service.exe" would be a prime target for us to use.

This process of path traversal can lead to security risks, as a malicious executable located in one of the directories along the way can be executed instead of the intended service. It's important to ensure that all services have properly quoted paths in the registry to prevent this type of vulnerability.

By monitoring the service start-up process in this way, you can gain a deeper understanding of how Unquoted Service Paths can lead to privilege escalation and other security risks. This information can help you make informed decisions about securing your own systems and applications.

Taking advantage of the vulnerability

In this example, a malicious service named "*by your command*" has been copied to "C:\Program Files\My Application Files" and renamed to "service.exe". This file is now in the path that Windows will search during path traversal when starting the unquoted service.

When the unquoted service is (re)started, Windows will perform a search and locate the "service.exe" file in "C:\Program Files\My Application Files". This file, which is actually the malicious "*by your command*" service, will then be executed with the same privileges as the system.

After placing the “Service.exe” in the directory and restarting the service, you can run Process Monitor at the same time to see that the behavior has changed. The line “C:\Program Files\My Application Files\Service.exe” is no longer listed as not found and the process will now run as expected.

services.exe	880	CreateFile	C:\Program	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program.exe	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program Files\My	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program Files\My.exe	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program Files\My Application	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program Files\My Application.exe	NAME NOT FOUND
services.exe	880	CreateFile	C:\Program Files\My Application Files\Service	NAME NOT FOUND

After the malicious service has been started, it will create a named pipe that can be used to run any command as system. Note that this is just part of the custom created code, and not available with each service. To locate this named pipe, you can use PowerShell to search for the pipe’s name.

Here’s an example of a PowerShell command that will search for the named pipe:

```
Get-ChildItem -Path "\\.\pipe\" | Where-Object {$_.Name -like "*byyourcommand*"}
```

PowerShell

This command will search for all items within the “\\.\pipe\” directory and display any items whose names contain “byyourcommand”. If the named pipe has been created successfully, it should show up in the list of results.

```
PS C:\Users\Administrator> Get-ChildItem -Path "\\.\pipe\" | Where-Object {$_.Name -like "by*"}

Directory: \\.\pipe

Mode                LastWriteTime         Length Name
-----
1-1-1601          01:00             1 byyourcommand
```

You can then use this named pipe to run any command as system, as long as you have the proper permissions to access the service. It’s important to stress that this creates a significant security vulnerability, as an attacker could use this named pipe to execute malicious code and gain full control over the system.

It’s important to note that this scenario could have been prevented if the original service had a properly quoted path in the registry, as Windows would have only checked the exact directory specified in the path and would not have performed any path traversal.

Executing the malicious code

With the named pipe now open, an attacker can send commands to the pipe to perform malicious actions on the system. For example, you could use the following command to

add a normal user named “Luke” to the local administrators group:

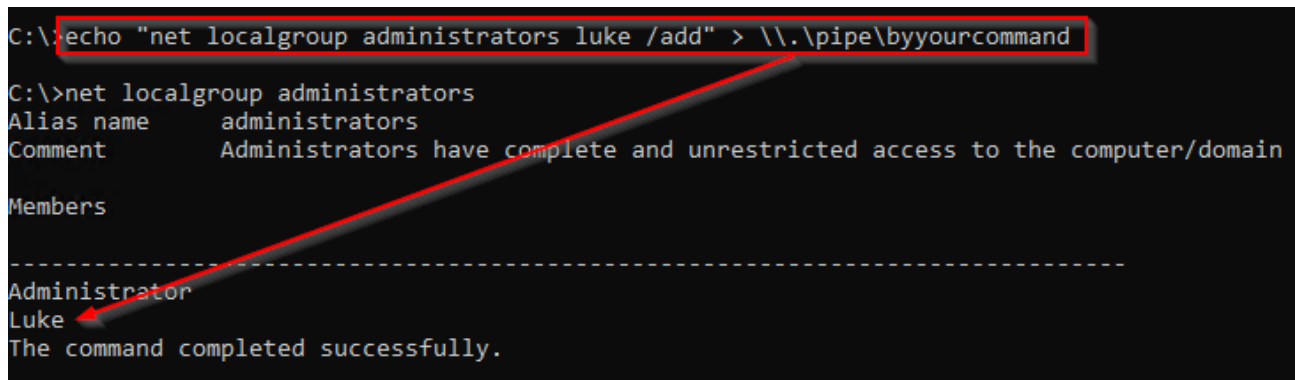
```
echo "net localgroup administrators luke /add" > \\.\pipe\byyourcommand
```

BAT (Batchfile)

This command, when run as the system, will add the user “Luke” to the local administrators group, giving them elevated privileges on the system. To verify the results, you can use the following command to display the members of the local administrators group:

```
net localgroup administrators
```

BAT (Batchfile)

A screenshot of a Windows command prompt window. The first line shows a command being executed: `C:\>echo "net localgroup administrators luke /add" > \\.\pipe\byyourcommand`. This line is highlighted with a red rectangular box. The second line shows the output of the command: `C:\>net localgroup administrators`. The output displays the details of the 'administrators' group, including its alias name, comment, and a list of members. The members list includes 'Administrator' and 'Luke'. A red arrow points from the 'Luke' entry in the members list to the 'Administrator' entry. The command prompt concludes with the message 'The command completed successfully.'

The consequences of this action could be severe, as the added user now has complete control over the system. They could install malicious software, steal sensitive information, or make changes to the system that could compromise its security. It is important to always be aware of the potential risks of running commands with elevated privileges, and to use them responsibly.

Dependencies for success

The exploitation of an unquoted service path vulnerability requires several dependencies to be met for success. Firstly, the service path in the registry must not be enclosed in quotes, which is the essence of this entire post, stick to the programming recommendations Microsoft provides. Secondly, the third level directory of the service must contain a space in its name, for example “*C:\Program Files\My Application Files\Service Files*”. Thirdly, the user must have write permissions to that directory, allowing them to place a malicious file there. Finally, the user must have the capability to send commands to the named pipe created by the service, which runs with system privileges.

It’s important to note that Windows Access Control Lists (ACLs) protect the file system and services alike, blocking normal users from copying files to protected areas or sending commands to the named pipe running as system. However, in the context of this demonstration, it’s also possible that the code from the malicious service could include the necessary commands to exploit the system. This is a reminder that the demonstration

serves only to highlight the dangers of Unquoted Service Paths and proper protection measures should always be taken to secure systems against such threats.

Protective measures

Preventing the exploitation of Unquoted Service Paths is an important step in securing Windows systems. This vulnerability can be used to elevate privileges and gain system-level access, leading to potential data breaches and other malicious activities. To mitigate the risk, it's essential to take a number of proactive steps to secure your systems and prevent unauthorized access. In this section, I've outlined a top 5 of actions you can take to prevent the misuse of Unquoted Service Paths and strengthen the security of your Windows environment.

1. Quote the Path in the Registry: Always ensure that the path in the registry for each service is properly quoted.
2. Restrict Write Access: Limit write access to the directories where the service executables are stored.
3. Monitor for Unusual Activity: Monitor your systems for unusual or unauthorized activity.
4. Keep Systems Up-to-Date: Regularly update your systems with the latest security patches to prevent potential exploitation of known vulnerabilities.
5. Use Application Whitelisting: Implement application whitelisting to prevent the execution of unauthorized applications, including malicious ones that could exploit the unquoted service path vulnerability.

Closing thoughts

In conclusion, it is important to take protective measures to prevent the misuse of the Unquoted Service Paths vulnerability. In this article, we discussed the specifics of this vulnerability and the demonstration of how it can be misused. To reiterate, it is crucial to follow the top 5 actions recommended to prevent exploitation of this and other vulnerabilities.

For those who would like to try the demonstration for themselves, the demo files used in this article can be [downloaded](#) for educational purposes only. However, it is important to note that these files are for demo purposes only and should not be used in a production environment as they could pose a security risk. Always use caution when handling sensitive files and systems.